



Bahria University, Islamabad

Department of Software Engineering

Computer Communication & Networking Lab

(Fall-2025)

Instructor: Engr. Saad Mazhar

Student: Aizaz Ahmed (01-131232-106)

Class: BSE 5-A

Lab Journal: 09

Date: 09-11-2025

Task No:	Task Wise Marks		Documentation Marks		Total Marks (20)
	Assigned	Obtained	Assigned	Obtained	
1					
2					
3					
4					
5					

LAB:09 COHESION AND COUPLING

Lab Tasks:

Task 1:

Discuss at least one scenario that clearly explains the concept of cohesion and coupling with reference to java. Explain the example with code.

- You can take of any real time scenario as a case study.
- You can also use your already developed projects (make some modifications through cohesion and coupling).

OUTPUT:

Scenario:

Let's take an **Online Shopping App**.

We'll design two classes — Product and Order.

Each class does its **own specific task** (high cohesion) and communicates using **loose coupling** through method calls, not direct access to data.

CODE:

```
class Product {  
  
    private String name;  
  
    private double price;  
  
  
    public Product(String name, double price) {  
  
        this.name = name;  
  
        this.price = price;  
  
    }  
  
  
    public double getPrice() {  
  
        return price;  
  
    }  
  
  
    public String getName() {
```

```
        return name;
    }
}

class Order {
    public double calculateTotal(Product product, int quantity) {
        return product.getPrice() * quantity;
    }
}
```

```
public class ShoppingApp {
    public static void main(String[] args) {
        Product p = new Product("T-Shirt", 1500.0);
        Order order = new Order();
        double total = order.calculateTotal(p, 3);

        System.out.println("Product: " + p.getName());
        System.out.println("Total Price: Rs. " + total);
    }
}
```

Task 2:

Is the code below tightly coupled if yes provide a reason and implement a loosely coupled working version of it?

```
// Java program to illustrate
// tight coupling concept
class Volume
{
    public static void main(String args[])
```

```

    {
        Box b = new Box(5,5,5);
        System.out.println(b.volume);
    }
}

class Box
{
    public int volume;
    Box(int length, int width, int height)
    {
        this.volume = length * width * height;
    }
}

```

OUTPUT:

REASON:

- Volume class directly accesses Box's internal variable volume.
- If the internal logic changes, Volume must also change → Tight Coupling

CODE:

LOOSELY COUPLED:

```

class Volume {

    public static void main(String args[]) {

        Box b = new Box(5,5,5);

        System.out.println("Volume: " + b.getVolume());

    }

}

```

```
class Box {  
    private int volume; // encapsulated  
  
    Box(int length, int width, int height) {  
        this.volume = length * width * height;  
    }  
  
    public int getVolume() {  
        return volume;  
    }  
}
```

Task 3:

Is the code below highly cohesive is yes kindly provide a reason?

```
class Name {  
    String name;  
    public String getName(String name)  
    {  
        this.name = name;  
        return name;  
    }  
}
```

```
class Age {  
    int age;  
    public int getAge(int age)
```

```
    {  
        this.age = age;  
        return age;  
    }  
}
```

```
class Number {  
    int mobileno;  
    public int getNumber(int mobileno)  
    {  
        this.mobileno = mobileno;  
        return mobileno;  
    }  
}
```

```
class Display {  
    public static void main(String[] args)  
    {  
        Name n = new Name();  
        System.out.println(n.getName("Geeksforgeeks"));  
        Age a = new Age();  
        System.out.println(a.getAge(10));  
        Number no = new Number();  
        System.out.println(no.getNumber(1234567891));  
    }  
}
```

```
class Name {  
    String name;
```

```
        public String getName(String name)
        {
            this.name = name;
            return name;
        }
    }
```

```
class Age {
    int age;
    public int getAge(int age)
    {
        this.age = age;
        return age;
    }
}
```

```
class Number {
    int mobileno;
    public int getNumber(int mobileno)
    {
        this.mobileno = mobileno;
        return mobileno;
    }
}
```

```
class Display {
    public static void main(String[] args)
    {
        Name n = new Name();
    }
}
```

```
        System.out.println(n.getName("Geeksforgeeks"));

        Age a = new Age();

        System.out.println(a.getAge(10));

        Number no = new Number();

        System.out.println(no.getNumber(1234567891));

    }

}
```

OUTPUT:

Yes, the code is highly cohesive.

Reason:

Each class has a **single, specific responsibility**:

- Name → handles name
- Age → handles age
- Number → handles mobile number
- Display → only displays data

Thus, each class performs **one well-defined task**, showing **High Cohesion**.